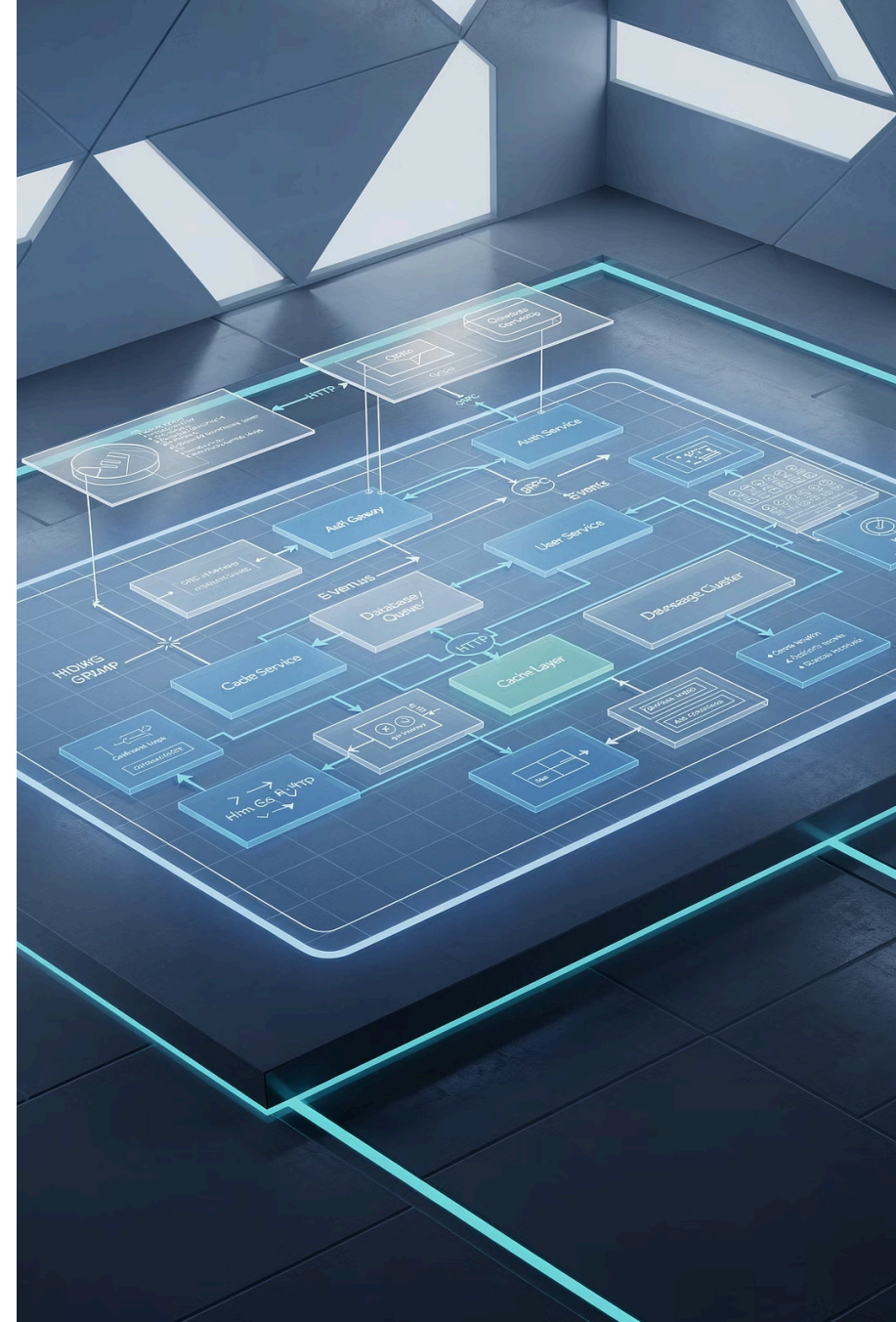


SOLID-Prinzipien im objektorientierten Design

Die SOLID-Prinzipien sind fünf grundlegende Regeln für gutes objektorientiertes Design. Sie helfen dabei, Programme **übersichtlich, verständlich, wartbar, erweiterbar und wiederverwendbar** zu gestalten.



Warum SOLID?

Gerade in größeren Programmen reicht es nicht aus, dass Code nur funktioniert. Entscheidend ist auch, **wie Klassen, Beziehungen und Verantwortlichkeiten aufgebaut sind**.

Die fünf SOLID-Prinzipien unterstützen genau dabei – sie geben klare Leitlinien für ein durchdachtes objektorientiertes Design.

S

Single Responsibility

O

Open/Closed

L

Liskov Substitution

I

Interface Segregation

D

Dependency Inversion

1. Single Responsibility Principle (SRP)

Eine Klasse soll genau eine klar abgegrenzte Hauptaufgabe besitzen.

Eine Klasse soll nicht gleichzeitig viele unterschiedliche Verantwortlichkeiten übernehmen. Wenn eine einzige Klasse Medien verwaltet, Druckfunktionen ausführt, Ausleihen speichert *und* Statusmeldungen erzeugt, wird sie schnell unübersichtlich.

Medium

Allgemeine Merkmale eines Mediums
beschreiben

Ausleihe

Die Ausleihbeziehung modellieren

Bibliothek

Medien verwalten

Das Prinzip hält Klassen kleiner und klarer, ermöglicht gezielte Änderungen und trennt Verantwortlichkeiten sauber.

SRP – Bezug zur Aufgabe

In der Aufgabenstellung wird das Single Responsibility Principle durch folgende Strukturen sichtbar – jede besitzt eine eigene, klar abgegrenzte Hauptaufgabe:



Medium

Gemeinsame Mediendaten bündeln



Person

Personendaten verwalten



Ausleihe

Die Ausleihbeziehung abbilden



Bibliothek

Verwaltung der Medien



Medienstatus

Zustände eines Mediums modellieren

2. Open/Closed Principle (OCP)

Softwarebausteine sollen offen für Erweiterungen, aber geschlossen für Änderungen sein.

Bestehende Klassen sollen nicht ständig umgebaut werden, wenn neue Anforderungen hinzukommen. Stattdessen werden neue Fälle durch **Erweiterungen** ergänzt.

Neue Unterklassen

Zeitschrift, EBook, Hoerbuch –
ergänzzbar ohne Umbau der Klasse
Medium

Neue Interfaces

Eine Klasse erhält zusätzliche
Fähigkeiten durch Implementierung
eines neuen Interfaces – ohne die
Gesamtstruktur umzubauen.



OCP – Bezug zur Aufgabe

Das Open/Closed Principle zeigt sich in der Aufgabe durch konkrete Erweiterungsmöglichkeiten, die das System offen und flexibel halten:

→ **Neue Unterklassen von Medium**

z. B. Zeitschrift, EBook, Hoerbuch –
ohne Änderung der Basisklasse

→ **Zusätzliche Enum-Werte in Medienstatus**

Neue Zustände ergänzbar ohne Umbau
bestehender Logik

→ **Weitere implementierende Klassen**

Für Interfaces wie Ausleihbar,
Druckbar oder Archivierbar

Dadurch bleibt das System **erweiterbar**, ohne bestehenden Code ständig anpassen zu müssen.

3. Liskov Substitution Principle (LSP)

Objekte einer Unterklasse sollen überall dort eingesetzt werden können, wo Objekte der Oberklasse erwartet werden.

Eine Unterklasse muss sich fachlich so verhalten, dass sie die Oberklasse sinnvoll ersetzt. Das Prinzip schützt vor fachlich unpassender Vererbung.

Buch ist ein Medium

Kann überall dort verwendet werden, wo Medium erwartet wird

DVD ist ein Medium

Ebenso korrekt über den Typ Medium ansprechbar

Polymorphie

Methoden wie `gibMedienInfo()` werden passend überschrieben – `List<Medium>` verarbeitet beide korrekt

4. Interface Segregation Principle (ISP)

Klassen sollen nicht gezwungen werden, Methoden zu implementieren, die sie nicht benötigen.

Besser: mehrere kleine, passende Interfaces statt eines großen Sammel-Interfaces.

Ein großes Interface

drucken(), archivieren(), ausleihen(),
streamen() – zu viele unpassende
Methoden für jede Klasse

Getrennte Interfaces

Druckbar – nur Druckverhalten

Archivierbar – nur Archivierung

Ausleihbar – nur Ausleihverhalten

Jede Klasse implementiert nur, was
wirklich zu ihr passt.



5. Dependency Inversion Principle (DIP)

Klassen sollen möglichst von Abstraktionen statt von konkreten Implementierungen abhängen.

Programme sollen mit allgemeinen Typen arbeiten, nicht nur mit ganz bestimmten Einzelklassen. Dadurch wird das System **flexibler und austauschbarer**.

List<Medium> statt List<Buch>

Die Bibliothek arbeitet mit dem abstrakten Typ Medium

Gegen Interfaces arbeiten

z. B. gegen Ausleihbar statt nur gegen Buch – flexibler und erweiterbar

Offenheit für Erweiterungen

Durch Abstraktionen bleibt die Modellierung offen für neue Klassen und Änderungen

Bedeutung der SOLID-Prinzipien für diese Aufgabe

Die SOLID-Prinzipien verbinden nicht nur technische Regeln mit Java-Syntax – sie unterstützen vor allem ein **durchdachtes objektorientiertes Design**.



Struktur & Trennung

Klassen sauber strukturieren und Verantwortlichkeiten klar trennen



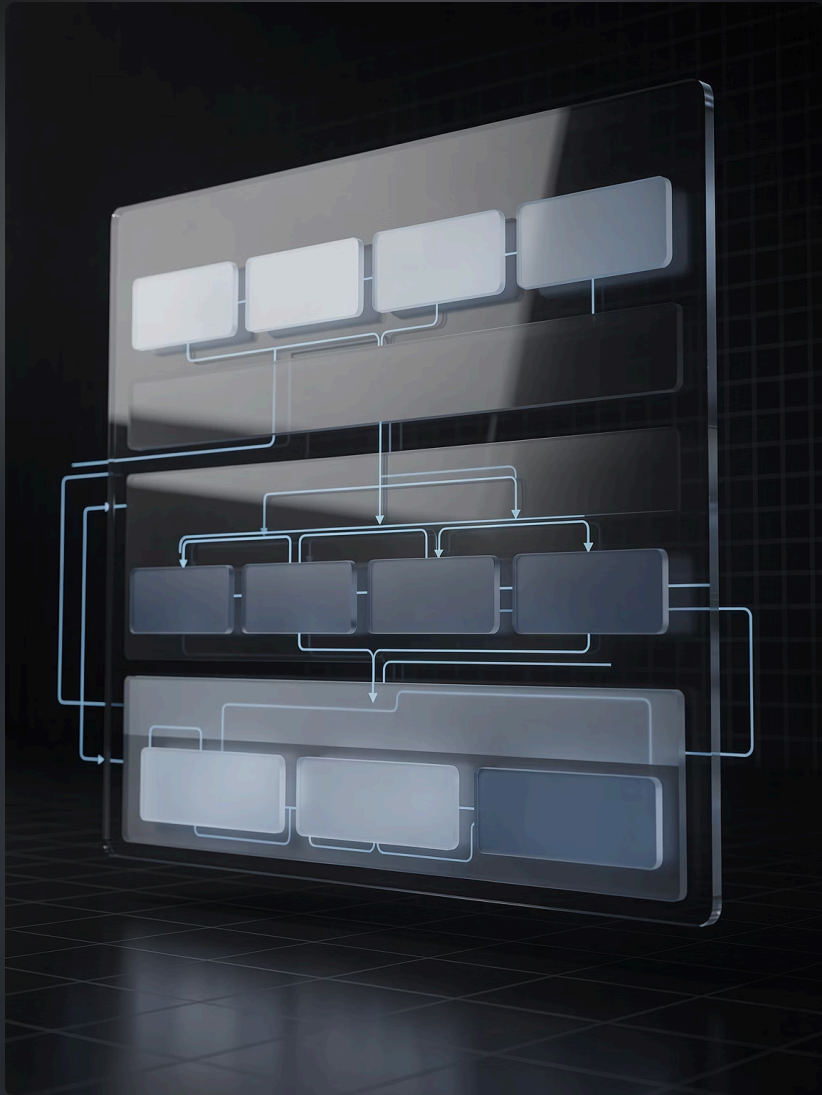
Vererbung & Interfaces

Vererbung sinnvoll einsetzen und Interfaces gezielt verwenden



Erweiterbarkeit

Erweiterbarkeit vorbereiten und die Gesamtstruktur fachlich begründen



Zusammenhang mit weiteren Themen der Aufgabe

Die SOLID-Prinzipien stehen in enger Verbindung zu den anderen Inhalten der Aufgabenstellung:



Enums

Modellieren feste Zustände sauber und klar – unterstützen eine eindeutige, wartbare Struktur.



Assoziationen

Saubere Beziehungen zwischen Klassen verhindern das Vermischen von Verantwortlichkeiten.



Vererbung

Muss fachlich sinnvoll sein – zentral für das Liskov Substitution Principle.



Abstrakte Klassen

Bündeln gemeinsames Verhalten und unterstützen Erweiterbarkeit.



Interfaces

Fördern lose Kopplung und kleine, spezialisierte Verträge – wichtig für ISP und DIP.

Merksätze zu SOLID & Lernziele

S – Single Responsibility

Eine Klasse soll genau eine Hauptaufgabe haben.

O – Open/Closed

Leicht erweiterbar, ohne bestehenden Code ständig umzubauen.

L – Liskov Substitution

Unterklassen müssen die Oberklasse fachlich sinnvoll ersetzen können.

I – Interface Segregation

Lieber mehrere kleine, passende Interfaces als ein großes Sammel-Interface.

D – Dependency Inversion

Abhängigkeiten sollen auf Abstraktionen beruhen, nicht auf konkreten Klassen.

- 📌 **Lernziel:** Verstehen, was die fünf SOLID-Prinzipien bedeuten, warum sie für OOP wichtig sind, wie sie mit Vererbung, Interfaces und Assoziationen zusammenhängen – und wie sie in einer Java-Modellierung praktisch erkennbar werden.